

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
дисциплины
«Искусственный интеллект и машинное обучение»

Выполнил:
Лебединский Даниил
Александрович 2 курс,
группа ИВТ-б-о-24-2,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники института
перспективной инженерии Воронкин
Р.А

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Цель работы: исследовать базовые возможности интерактивных оболочек с Jupyter Notebook, JupyterLab и Google Colab для языка программирования Python.

Ссылка на репозиторий: <https://github.com/FomalhautSystem/AI-LAB-1>

Конспект теоретического материала:

IPython — мощный инструмент для работы с Python. Его базовые компоненты — это интерактивная оболочка с широким набором возможностей и ядро для Jupyter. Jupyter Notebook, в свою очередь, представляет собой графическую веб-оболочку для IPython, развивающую идею консольного подхода к интерактивным вычислениям.

Ключевые особенности платформы: комплексная интроспекция объектов, сохранение истории ввода между сессиями, кэширование результатов, расширяемая система «магических» команд, логирование сессий, дополнительный командный синтаксис, подсветка кода, доступ к системной оболочке, а также интеграция с отладчиком pdb и профайлером Python.

IPython позволяет подключать несколько клиентов к одному вычислительному ядру и благодаря своей архитектуре может работать в составе параллельного кластера. В Jupyter Notebook можно разрабатывать, документировать и выполнять приложения на Python. Он состоит из двух компонентов: веб-приложения, запускаемого в браузере, и ноутбуков — файлов, в которых можно работать с исходным кодом программы, запускать его, вводить и выводить данные и т. п.

Ход работы:

Для выполнения данной практической работы был изучен теоретический материал и были проработаны примеры, для этого каждый пример был перенесён в интерактивную оболочку Google Colab и общий файл всех примеров был сохранен в репозиторий.

В первом задании необходимо освоить работу с типами ячеек и форматированием текста. Для этого была создана Markdown-ячейка, имеющая списки заголовки и индивидуальное задание:

Практическое задание №1

Пример форматирования текста

****Это жирный текст****

Это курсивный текст

*****Это жирный курсив*****

Нумерованный список:

1. Первый пункт
2. Второй пункт
3. Третий пункт

Маркированный список:

- Пункт один
- Пункт два
- Пункт три

Ряд Маклорена для синуса

\$\$

$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$

\$\$

Имеет следующий вид:

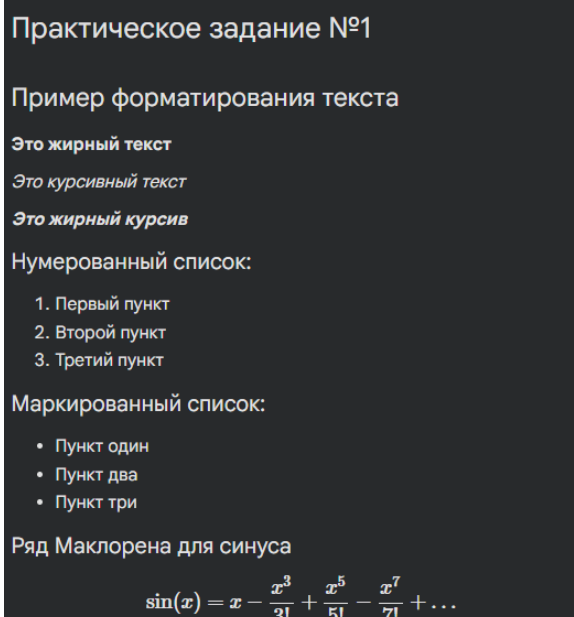


Рисунок 1. Выполнение ПЗ1

Далее следует задание на изучение загрузки, создания и сохранение файлов в Google Colab:

```
import os

with open("Venochek.txt", "w") as file:
    file.write("И плывет веночек вдоль ночной воды\n")
    file.write("Одинокая стою на берегу\n")
    file.write("Вижу отражение своей мечты\n")
    file.write("Но найти её на небе не могу\n")

print("Файл создан и данные записаны.\n")

with open("Venochek.txt", "r") as file:
    content = file.read()

print("Содержимое файла:")
print(content)

if os.path.exists("Venochek.txt"):
    print("Файл существует.")
else:
    print("Веночек не найден.")

os.remove("Venochek.txt")
print("Веночек удалён.")
```



```
import os

with open("Venochek.txt", "w") as file:
    file.write("И плывет веночек вдоль ночной воды\n")
    file.write("Одинокая стою на берегу\n")
    file.write("Вижу отражение своей мечты\n")
    file.write("Но найти её на небе не могу\n")

print("Файл создан и данные записаны.\n")

with open("Venochek.txt", "r") as file:
    content = file.read()

print("Содержимое файла:")
print(content)

if os.path.exists("Venochek.txt"):
    print("Файл существует.")
else:
    print("Веночек не найден.")

os.remove("Venochek.txt")
print("Веночек удалён.")
```

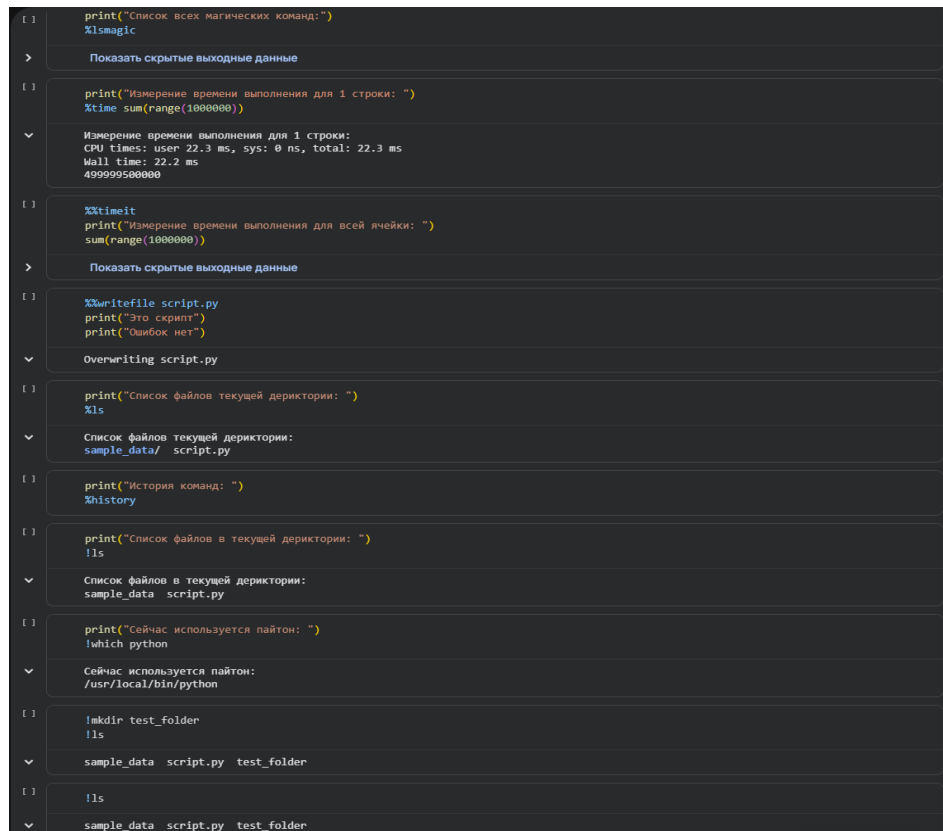
*** Файл создан и данные записаны.

Содержимое файла:
И плывет веночек вдоль ночной воды
Одинокая стою на берегу
Вижу отражение своей мечты
Но найти её на небе не могу

Файл существует.
Веночек удалён.

Рисунок 2. Выполнение ПЗ2

Были выполнены задания на изучение магических команд, взаимодействие с оболочкой системы.



```
[ ]: print("Список всех магических команд:")
%lsmagic
> Показать скрытые выходные данные

[ ]: print("Измерение времени выполнения для 1 строки: ")
%time sum(range(1000000))
Измерение времени выполнения для 1 строки:
CPU times: user 22.3 ms, sys: 0 ns, total: 22.3 ms
Wall time: 22.2 ms
4999995000000

[ ]: %timeit
print("Измерение времени выполнения для всей ячейки: ")
sum(range(1000000))
> Показать скрытые выходные данные

[ ]: %writefile script.py
print("Это скрипт")
print("Ошибок нет")
> Overwriting script.py

[ ]: print("Список файлов текущей дериктории: ")
%ls
> Список файлов текущей дериктории:
sample_data/ script.py

[ ]: print("История команд: ")
%history

[ ]: print("Список файлов в текущей дериктории: ")
!ls
> Список файлов в текущей дериктории:
sample_data script.py

[ ]: print("Сейчас используется пайтон: ")
!which python
> Сейчас используется пайтон:
/usr/local/bin/python

[ ]: mkdir test_folder
!ls
> sample_data script.py test_folder

[ ]: !ls
> sample_data script.py test_folder
```

Рисунок 3. Выполнение заданий

В последнем задании необходимо подключить Google Drive, сохранить туда текстовый файл, прочитать файл из Google Drive и создать CSV-файл.

Код выполнения последнего задания:

```
from google.colab import drive
```

```
drive.mount('/content/drive')
```

```
file_path = "/content/drive/MyDrive/my_text_file.txt"
```

```
with open(file_path, "w") as f:
```

```
    f.write("Это тестовый файл, сохраненный в Google Drive.\n")
```

```

f.write("Вторая строка текста.")

print ("Файл успешно сохранен в Google Drive.")

with open(file_path, "r") as f:
    content = f.read()
    print("Содержимое файла:\n", content)

students = [
["Каиров В.С.", 17, "ИВТ-241"],
["Грабарь А.П.", 19, "ИВТ-241"],
["Щегольков С.И.", 19, "ИВТ-241"]
]

csv_path = "/content/drive/MyDrive/students.csv"
with open(csv_path, "w") as f:
    for student in students:
        f.write(",".join(map(str, student)) + "\n")

print("Файл students.csv успешно сохранен в Google Drive.")

```



```

from google.colab import drive

drive.mount('/content/drive')

file_path = "/content/drive/MyDrive/my_text_file.txt"
with open(file_path, "w") as f:
    f.write("Это тестовый файл, сохраненный в Google Drive.\n")
    f.write("Вторая строка текста.")

print ("Файл успешно сохранен в Google Drive.")

with open(file_path, "r") as f:
    content = f.read()
    print("Содержимое файла:\n", content)

students = [
["Каиров В.С.", 17, "ИВТ-241"],
["Грабарь А.П.", 19, "ИВТ-241"],
["Щегольков С.И.", 19, "ИВТ-241"]
]

csv_path = "/content/drive/MyDrive/students.csv"
with open(csv_path, "w") as f:
    for student in students:
        f.write(",".join(map(str, student)) + "\n")

print("Файл students.csv успешно сохранен в Google Drive.")

```

Рисунок 4. Выполненное последнее задание

Все задания и примеры были закомичены и сохранены в удаленный репозиторий на GitHub.

Контрольные вопросы:

1) Какие основные отличия JupyterLab от Jupyter Notebook?

Jupyter Notebook — это классическая веб-оболочка, где всё построено вокруг одного документа: страница с ячейками, минимум панелей, отдельные вкладки браузера для разных ноутбуков. JupyterLab — это следующее поколение той же среды: интерфейс построен как гибкое рабочее пространство с вкладками, перетаскиваемыми панелями и сплит-экраном, где одновременно можно открыть ноутбук, терминал, текстовый редактор, файловый менеджер и просмотрщик данных. JupyterLab поддерживает расширения, тёмную тему, работу с несколькими документами в одном окне и более удобную навигацию по файловой системе. Jupyter Notebook проще и легче, JupyterLab — мощнее и ближе к полноценной IDE. При этом оба используют одно и то же ядро IPython и формат .ipynb, поэтому ноутбуки совместимы.

2) Как создать новую рабочую среду в JupyterLab?

В JupyterLab есть два смысла «рабочей среды»:

Workspace — сохранённая раскладка интерфейса (какие вкладки и панели открыты). Создаётся через File → Workspaces → New Workspace, или через боковую панель «Running Terminals and Kernels» → раздел Workspaces, или просто переходом по URL /lab/workspaces/имя. Управление: create, clone, rename, reset, delete.

Ядро (kernel) — изолированное Python-окружение с нужными пакетами.

Порядок:

Открыть терминал в JupyterLab (File → New → Terminal).

Создать окружение:

venv: `python -m venv ~/.venv-testing → source ~/.venv-testing/bin/activate`

conda: `conda create --name test-env python=3.11 ipykernel` → `conda activate test-env`

Установить и зарегистрировать ядро: `pip install ipykernel`, затем `python -m ipykernel install --user --name "myenv" --display-name "Python (myenv)"`

Проверить: `jupyter kernelspec list`.

Обновить страницу JupyterLab — новое ядро появится в Launcher.

3) Какие типы ячеек поддерживаются в JupyterLab и как их переключать?

В JupyterLab поддерживаются три основных типа ячеек:

Code — для исполняемого кода (Python и другие языки через магические команды).

Markdown — для текста с форматированием: заголовки, списки, ссылки, формулы LaTeX, картинки.

Raw — «сырой» текст без обработки; используется редко, например для вставки готового кода или форматов, которые не должны рендериться.

Как переключать:

Через выпадающий список в панели инструментов ноутбука (там отображается текущий тип ячейки).

Через меню `Run` → `Cell Type` → выбрать `Code` / `Markdown` / `Raw`.

Через горячие клавиши в командном режиме (ячейка выделена, но не в режиме редактирования):

Y — переключить в Code

M — переключить в Markdown

R — переключить в Raw

Через палитру команд (`Ctrl+Shift+C` или иконка палитры) — поиск по «cell type».

Чтобы войти в командный режим, нажми `Esc`; для редактирования содержимого ячейки — `Enter`.

4) Как выполнить код в ячейке и какие горячие клавиши для этого используются?

Выполнить код в ячейке можно несколькими способами:

Через кнопку Run (иконка воспроизведения) в панели инструментов ноутбука.

Через меню Run → Run Selected Cells.

Горячими клавишами:

Shift+Enter — выполнить ячейку и перейти к следующей.

Ctrl+Enter — выполнить ячейку и остаться в ней.

Alt+Enter — выполнить ячейку и вставить новую ниже.

Ctrl+Shift+Enter (или Ctrl+Alt+Enter в новых версиях) — выполнить все ячейки ноутбука.

Через палитру команд (Ctrl+Shift+C) — поиск по «run cell».

5) Как запустить терминал или текстовый редактор внутри JupyterLab?

Терминал:

Через Launcher — на стартовой странице в разделе Other нажать Terminal.

Через меню File → New → Terminal.

Горячей клавишей в командном режиме ноутбука: нажать Ctrl+Shift+L (в некоторых сборках работает иконка терминала на левой панели).

Терминал открывается как вкладка внутри JupyterLab и работает с той же файловой системой, что и ноутбуки.

Текстовый редактор:

Через Launcher — в разделе Other нажать Text Editor (или Editor).

Через меню File → New → Text File.

Двойным кликом по любому текстовому файлу в файловом менеджере слева (например, .py, .txt, .md, .json).

Через контекстное меню файла в File Browser → Open With → Editor.

Редактор поддерживает подсветку синтаксиса, поиск, отступы и сохранение файлов прямо из JupyterLab, без внешних программ.

6) Какие инструменты JupyterLab позволяют работать с файлами и структурами каталогов?

File Browser — основная левая панель для работы с файлами и папками.

Позволяет создавать, переименовывать, удалять, копировать, перемещать

файлы и каталоги, загружать файлы с локального компьютера и скачивать их из JupyterLab. Через контекстное меню доступны операции New File, New Folder, Upload, Download, Rename, Duplicate, Delete, Copy Path, Open With.

Launcher — стартовая страница для быстрого создания новых файлов: ноутбуков, текстовых файлов, Markdown-файлов, терминалов, консолей.

Text Editor — встроенный редактор для открытия и изменения текстовых файлов (код, конфиги, разметка).

Main Area (рабочая область) — открытые файлы и папки отображаются как вкладки; можно открывать несколько файлов одновременно и перетаскивать вкладки для организации пространства.

Terminal — позволяет работать с файловой системой через командную строку: ls, cd, mkdir, rm, cp, mv и т. д.

Running Terminals and Kernels — панель, показывающая активные терминалы, ядра и вкладки; помогает управлять запущенными сессиями.

Command Palette — позволяет находить команды работы с файлами (например, «New Folder», «Upload File») без поиска по меню.

File Editor / CSV Viewer / Image Viewer — специализированные просмотрщики: для CSV, изображений и других типов файлов открываются соответствующие вьюеры.

Path navigation (хлебные крошки) — вверху File Browser отображается текущий путь, по которому можно перемещаться вверх по каталогам.

7) Как можно управлять ядрами в JupyterLab?

Выбор ядра для ноутбука:

Через выпадающий список ядер в правом верхнем углу открытого ноутбука.

Через меню Kernel → Change Kernel для переключения на другое ядро.

При создании нового ноутбука — в Launcher выбирается нужное ядро из списка доступных.

Управление запущенными ядрами:

Панель Running Terminals and Kernels (левая боковая панель) показывает активные ядра и позволяет их останавливать.

Через меню Kernel доступны команды Interrupt, Restart, Restart & Clear Output, Shut Down Kernel.

Регистрация новых ядер:

Виртуальные окружения (venv, conda) становятся доступны как ядра после установки в них пакета ipykernel.

Зарегистрировать окружение как ядро можно командой `python -m ipykernel install --user --name myenv --display-name "Python (myenv)"` в терминале JupyterLab.

Существуют расширения (например, nb_venv_kernels), которые автоматически находят и регистрируют окружения в селекторе ядер.

8) Каковы основные возможности системы вкладок и окон в интерфейсе JupyterLab?

Вкладки:

Каждый открытый файл, ноутбук, терминал или консоль открывается как отдельная вкладка в рабочей области.

Вкладки можно перетаскивать, менять их порядок, закрывать и открывать заново.

Одновременно можно держать открытыми много документов и переключаться между ними кликом по вкладке.

Разделение рабочей области (split view):

Рабочую область можно разделить на несколько панелей по горизонтали или

вертикали.

Перетаскивая вкладку к краю окна, можно создать новую панель и разместить документы рядом.

Это позволяет, например, держать ноутбук и терминал или два ноутбука одновременно перед глазами.

Перетаскивание и закрепление панелей:

Вкладки и панели можно перетаскивать мышью в любое место интерфейса.

Боковые панели (File Browser, Running Terminals and Kernels, Table of Contents и др.) можно закреплять слева или справа, сворачивать и разворачивать.

Управление через меню View:

В меню View доступны команды для работы с окнами: Split Notebook Left/Right/Top/Bottom, Single Document Mode, Toggle Sidebar, Toggle Header, Presentation Mode.

Single Document Mode оставляет только одну активную вкладку, скрывая остальные.

Сохранение раскладки (Workspaces):

Текущее расположение вкладок и панелей можно сохранить как workspace.

При следующем запуске JupyterLab раскладка восстанавливается через File → Workspaces.

Горячие клавиши:

Переключение между вкладками, закрытие вкладки, деление панели — всё доступно через Command Palette и назначаемые горячие клавиши.

9) Какие магические команды можно использовать в JupyterLab для измерения времени выполнения кода? Приведите примеры.

Для измерения времени выполнения в IPython/JupyterLab используются две основные магические команды: `%timeit` и `%time`. Обе работают в JupyterLab так же, как в IPython.

`%timeit` — измеряет среднее время выполнения с автоматическим подбором числа прогонов, чтобы результат был статистически надёжным. Работает и как линейная, и как ячеечная команда.

Примеры:

```
%timeit sum(range(1000000))
```

```
%timeit -n 100 -r 5 [x**2 for x in range(1000)]
```

Линейная форма `%timeit` измеряет одну строку. Ячеечная форма `%%timeit` измеряет всю ячейку:

```
%%timeit
```

```
total = 0
```

```
for i in range(100000):
```

```
total += i
```

`%time` — измеряет время однократного выполнения строки или ячейки без повторных прогонов. Полезно, когда код долгий или имеет побочные эффекты.

Примеры:

```
%time sum(range(1000000))
```

Ячеечная форма:

```
%%time
```

```
result = []
```

```
for i in range(100000):
```

```
result.append(i**2)
```

Дополнительно:

`%%capture` — не для измерения времени, но часто используется вместе с ним: перехватывает вывод ячейки, чтобы отделить его от результатов замеров.

В Google Colab аналогичные команды работают так же, а для более сложных замеров можно использовать `%%timeit` с флагами `-n` (число прогонов) и `-r` (число повторов).

10) Какие магические команды позволяют запускать код на других языках программирования в JupyterLab?

В JupyterLab для запуска кода на других языках используются ячейечные магические команды, начинающиеся с двойного процента (`%%`). Они работают в IPython-ядре и передают содержимое ячейки соответствующему интерпретатору.

Основные команды:

`%%bash` — выполняет код на языке `bash` (оболочка Linux/macOS).

%%sh — синоним для %%bash в некоторых сборках.

%%html — выводит HTML-разметку прямо в ячейке.

%%javascript или %%js — исполняет JavaScript-код в браузере.

%%latex — рендерит LaTeX-разметку.

%%markdown — выводит текст как Markdown.

%%perl — выполняет код на Perl.

%%ruby — выполняет код на Ruby.

%%python2 — запускает код на Python 2 (если он установлен).

%%python3 — запускает код на Python 3.

%%R — выполняет код на языке R (требуется установка rpy2).

%%sql — выполняет SQL-запросы (требуется соответствующее расширение, например ipython-sql).

%%script — универсальная команда для запуска любого внешнего интерпретатора, например:

```
%%script node
```

```
console.log("Hello from Node.js")
```

Примеры:

```
%%bash
```

```
echo "Текущая дата:"
```

```
date
```

```
%%html
```

```
<h2>Заголовок</h2> <p>Это HTML-текст внутри ноутбука.</p>
```

```
%%javascript
```

```
alert("Hello from JavaScript");
```

Важно: команды с одним процентом (%) действуют на одну строку, команды с двумя процентами (%%) — на всю ячейку. Для работы некоторых из них (R, SQL, Perl, Ruby) нужно установить соответствующие пакеты или ядра.

11) Какие основные отличия Google Colab от JupyterLab?

Среда запуска:

Google Colab — облачный сервис, работает в браузере, код выполняется на серверах Google. Ничего устанавливать локально не нужно.

JupyterLab — локальная среда, запускается на своём компьютере или сервере, требует установки Python, JupyterLab и зависимостей.

Аппаратные ресурсы:

Google Colab предоставляет бесплатный доступ к GPU (Tesla T4 и др.) и TPU, а также платные тарифы с более мощными ресурсами.

JupyterLab использует только ресурсы машины, на которой запущен, — если нет своего GPU, вычисления идут на CPU.

Хранение файлов:

Google Colab хранит файлы временно: после завершения сессии они удаляются, если не сохранены на Google Drive или не скачаны.

JupyterLab сохраняет файлы на диск локально — они остаются между сессиями.

Интеграция:

Google Colab тесно интегрирован с Google Drive, Google Sheets, GitHub, Kaggle и другими сервисами.

JupyterLab — самостоятельная среда, интеграции настраиваются вручную (расширения, ядра, git).

Установка библиотек:

В Google Colab библиотеки ставятся прямо в ячейке через `!pip install` и работают до конца сессии.

В JupyterLab библиотеки ставятся в текущее окружение (venv, conda) и сохраняются между сессиями.

Интерфейс:

Google Colab — упрощённый, одна страница с ячейками, минимум настроек, ориентирован на быстрый старт.

JupyterLab — полноценная IDE: вкладки, панели, терминал, файловый менеджер, редактор, расширения.

Совместная работа:

Google Colab поддерживает одновременное редактирование ноутбука несколькими пользователями (как в Google Docs), комментарии и шаринг по ссылке.

JupyterLab по умолчанию однопользовательский; для совместной работы нужны JupyterHub, Voilà или сторонние решения.

Сохраняемость сессии:

В Google Colab сессия завершается после простоя (обычно через 90 минут) или через 12 часов непрерывной работы.

В JupyterLab сессия живёт, пока запущен сервер.

Формат файлов:

Оба используют один и тот же формат `.ipynb`, поэтому ноутбуки переносимы между Colab и JupyterLab.

12) Как создать новый ноутбук в Google Colab?

Через главную страницу Colab:

Открыть colab.research.google.com.

Войти в Google-аккаунт.

Нажать New notebook (или File → New notebook).

Откроется пустой ноутбук с одной Code-ячейкой.

Через Google Drive:

Открыть drive.google.com.

Нажать New → More → Google Colaboratory.

Если пункта нет — подключить Colab через «Connect more apps» и найти Google Colaboratory.

Из уже открытого Colab:

Меню File → New notebook.

Или File → New → New Python 3 notebook.

Через прямую ссылку:

Открыть colab.research.google.com/notebooks/intro.ipynb и нажать File → New notebook.

Горячие клавиши:

В открытом Colab: Ctrl+M, затем N — создать новый ноутбук (в некоторых раскладках Ctrl+Alt+N).

После создания:

Ноутбук автоматически сохраняется в папку Colab Notebooks на Google Drive.

Переименовать: клик по имени вверху или File → Rename.

Формат — .ipynb, совместим с JupyterLab.

13) Какие типы ячеек доступны в Google Colab, и как их переключать?

В Google Colab доступны три основных типа ячеек:

Code — для исполняемого кода на Python (и других языках через магические команды).

Text (Markdown) — для форматированного текста: заголовки, списки, ссылки, формулы LaTeX, изображения.

Section — заголовок раздела, который создаёт сворачиваемый блок в ноутбуке (в других средах аналога нет).

Как переключать:

Через выпадающий список в панели инструментов ячейки — там отображается текущий тип (Code или Text), рядом есть кнопки + Code и + Text для добавления новых ячеек.

Через меню Insert → Code cell / Text cell.

Через палитру команд (Ctrl+Shift+P) — поиск по cell type.

Горячими клавишами в командном режиме (ячейка выделена, но не редактируется):

Ctrl+M, затем Y — переключить в Code.

Ctrl+M, затем M — переключить в Text (Markdown).

Ctrl+M, затем H — открыть список всех горячих клавиш.

Переключение между режимами:

Esc — командный режим (для горячих клавиш).

Enter — режим редактирования ячейки.

14) Как выполнить код в ячейке Google Colab и какие горячие клавиши для этого используются?

Выполнить код в ячейке можно несколькими способами:

Через кнопку Run (иконка воспроизведения) слева от ячейки.

Через меню Runtime → Run all или Runtime → Run before / Run after.

Горячими клавишами:

Ctrl+Enter — выполнить ячейку и остаться в ней.

Shift+Enter — выполнить ячейку и перейти к следующей.

Alt+Enter — выполнить ячейку и вставить новую ниже.

Ctrl+Shift+Enter — выполнить все ячейки ноутбука.

Ctrl+F9 — выполнить все ячейки (в некоторых раскладках).

Ctrl+M, затем B — вставить новую ячейку ниже (не выполнение, но часто используется вместе).

Во время выполнения рядом с ячейкой появляется индикатор (вращающийся круг), после завершения — номер выполнения [1], [2] и т. д. Если ячейка выполняется долго, можно прервать через кнопку Stop или Runtime → Interrupt execution.

15) Какие способы загрузки и сохранения файлов поддерживает Google Colab?

Загрузка файлов в Colab:

Через панель Files (левая боковая панель) — нажать на иконку папки, затем кнопку Upload, выбрать файлы с локального компьютера. Также можно просто перетащить файлы в область /content.

Через код с `files.upload()` — открывается диалог выбора файла:

```
from google.colab import files
```

```
files.upload()
```

Загруженные файлы появляются в /content и возвращаются в виде словаря с именами и содержимым.

Через `wget` или `requests` — для скачивания файлов по ссылке:

```
!wget https://example.com/data.csv
```

Сохранение файлов:

Скачать на локальный компьютер — через контекстное меню файла в панели Files → Download, или кодом:

```
from google.colab import files  
files.download('model.pkl')
```

Файл сразу скачивается браузером.

Сохранить на Google Drive — смонтировать диск:

```
from google.colab import drive  
drive.mount('/content/drive')
```

После этого всё, что сохраняется по пути /content/drive/MyDrive/, остаётся навсегда, а не только на время сессии.

Сохранить ноутбук — File → Save (Ctrl+S), File → Download → .ipynb или .py, либо Save a copy in GitHub/Drive.

16) Как можно подключить Google Drive к Google Colab и работать с файлами?

Подключение Google Drive к Colab выполняется через монтирование файловой системы. После этого файлы Drive становятся доступны по обычным путям, как если бы они лежали локально.

Базовое монтирование:

```
from google.colab import drive  
drive.mount('/content/drive')
```

При первом запуске этой команды Colab запросит разрешение на доступ к вашему Google Drive. Нужно перейти по появившейся ссылке, выбрать аккаунт, разрешить доступ и скопировать сгенерированный код авторизации обратно в поле ввода в ноутбуке .

После монтирования ваш диск отображается в панели Files слева, а доступ к файлам осуществляется по пути:

```
/content/drive/MyDrive/имя_папки/имя_файла
```

Например, чтобы прочитать текстовый файл:

```
with open('/content/drive/MyDrive/Colab Notebooks/book.txt', 'r') as file:  
    content = file.read()  
    print(content[:500])
```

Или загрузить CSV в pandas:

```
import pandas as pd  
df = pd.read_csv('/content/drive/MyDrive/data/myData.csv')
```

Работа с общими дисками (Shared Drives):

Если файл находится на общем диске, к которому у вас есть доступ, сначала нужно создать ярлык на этот файл или папку в вашем личном Drive, чтобы он стал виден в Colab . После этого можно перейти в раздел общих дисков:

```
import os  
os.chdir("/content/drive/SharedDrives/")  
os.listdir()
```

Проверка существования пути:

Перед чтением файлов полезно убедиться, что папка или файл действительно доступны:

```
import os
```

```
path = '/content/drive/MyDrive/Практикум_3_семестр/handwritten-ocr'  
print(os.path.exists(path))
```

Копирование файлов из Drive в Colab:

Если нужно работать с файлами локально в Colab (например, для ускорения чтения), их можно скопировать из смонтированного диска во временную папку /content:

```
!cp -av '/content/drive/MyDrive/test-colab' '/content/'
```

17) Какие команды используются для загрузки файлов в Google Colab из локального компьютера?

Основной способ — через модуль files из библиотеки google.colab:

```
from google.colab import files  
uploaded = files.upload()
```

После выполнения открывается диалоговое окно выбора файла. Можно выбрать один или несколько файлов. Они загружаются в текущую директорию Colab (обычно /content), а функция возвращает словарь вида {имя_файла: содержимое_в_байтах}.

Загрузка одного файла и чтение его в pandas:

```
from google.colab import files  
uploaded = files.upload()  
  
import pandas as pd  
import io  
df = pd.read_csv(io.BytesIO(uploaded['data.csv']))
```

Загрузка нескольких файлов сразу:

```
from google.colab import files  
uploaded = files.upload()  
  
for filename in uploaded:  
    print(filename, len(uploaded[filename]), 'байт')
```

Через панель Files (без кода):

Слева открыть панель Files (иконка папки).

Нажать кнопку Upload.

Выбрать файлы на локальном компьютере — они появятся в /content.

Также можно просто перетащить файлы мышью в область файлов.

Через wget для скачивания по ссылке (не с локального компьютера, а из интернета):

```
!wget https://example.com/data.zip
```

Через git clone, если файлы лежат в репозитории:

```
!git clone https://github.com/username/repo.git
```

18) Как посмотреть список файлов, хранящихся в среде Google Colab?

Основные способы:

Панель Files (левая боковая панель) — нажать на иконку папки слева.

Откроется файловый менеджер, где видно все файлы и папки в /content, а также смонтированный Google Drive. Там же можно обновить список кнопкой Refresh.

Команда ls в ячейке:

```
!ls
```

или

```
!ls -la
```

Выводит список файлов текущей директории (обычно /content). Флаг -la показывает подробную информацию, включая скрытые файлы.

Через Python:

```
import os
```

```
os.listdir('/content')
```

Возвращает список имён файлов и папок в указанной директории.

С подробной информацией через pathlib:

```
from pathlib import Path
for p in Path('/content').iterdir():
    print(p.name, p.stat().st_size, 'байт')
```

Список файлов на Google Drive после монтирования:

```
!ls /content/drive/MyDrive
```

Или:

```
import os
os.listdir('/content/drive/MyDrive')
```

Проверка конкретного пути:

```
!ls /content/drive/MyDrive/Colab\ Notebooks
```

Навигация по директориям:

```
%cd /content/drive/MyDrive
!ls
```

19) Какие магические команды можно использовать в Google Colab для измерения времени выполнения кода? Приведите примеры.

В Google Colab, как и в IPython/JupyterLab, для замера времени используются две основные магические команды: `%timeit` и `%time`.

`%timeit` — измеряет среднее время выполнения с автоматическим подбором числа прогонов. Подходит для коротких операций.

Линейная форма — для одной строки:

```
%timeit sum(range(1000000))
```

С указанием числа прогонов и повторов:

```
%timeit -n 100 -r 5 [x**2 for x in range(1000)]
```

Ячеечная форма — для всего блока кода:

```
%%timeit
```

```
total = 0
```

```
for i in range(100000):
```

```
total += i
```

%time — измеряет время однократного выполнения. Удобно для долгих операций или кода с побочными эффектами.

Линейная форма:

```
%time sum(range(1000000))
```

Ячеечная форма:

```
%%time
```

```
result = []
```

```
for i in range(100000):
```

```
result.append(i**2)
```

20) Как можно изменить аппаратные ресурсы в Google Colab (например переключиться на GPU)?

Основной способ — через меню:

В верхнем меню Colab выбрать Runtime → Change runtime type.

В открывшемся окне в разделе Hardware accelerator выбрать нужный ускоритель:

None — только CPU (стандартный режим).

T4 GPU — бесплатный GPU NVIDIA Tesla T4, подходит для большинства задач глубокого обучения.

A100 GPU — более мощный GPU, доступен в платных тарифах.

L4 GPU — GPU для инференса и обработки видео.

TPU v2 — тензорный процессор Google для TensorFlow-задач.

Нажать Save — среда выполнения перезапустится с новыми ресурсами.

Через кнопку подключения:

Нажать кнопку Connect в правом верхнем углу ноутбука.

Выбрать Change runtime type.

Указать T4 GPU в качестве аппаратного ускорителя и сохранить.

Проверка, что GPU активен:

```
import tensorflow as tf
print(tf.config.list_physical_devices('GPU'))
```

Или для PyTorch:

```
import torch
print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'None')
```

Если вывод показывает GPU-устройство — всё работает

Вывод:

В ходе лабораторной работы были изучены базовые возможности интерактивных оболочек Jupyter Notebook, JupyterLab и Google Colab для языка Python. Освоены основные принципы работы с IPython: создание и переключение типов ячеек, форматирование текста в Markdown, выполнение кода и работа с «магическими» командами. Отработаны практические навыки загрузки, создания, чтения, сохранения и удаления файлов как в локальной среде Google Colab, так и через подключение Google Drive, включая создание CSV-файлов. Полученные результаты закоммичены в удалённый репозиторий на GitHub. Таким образом, цели работы достигнуты, а рассмотренные инструменты подтвердили свою эффективность для интерактивной разработки, документирования и выполнения приложений на Python.